

alterspec: the essential guide

How a product spec is started, tuned, designed, changed and handed over with alterspec, and which parts of that work are deterministic (the CLI) and which are done by the model (Claude Code skills and agents).

1. Starting a new project

- 1 In the project root: `npm install --save-dev @alterset/alterspec` then `npx alterspec init --name "My Product"`. This writes three folders, all meant to be committed: `spec/` (yours), `.alterspec/` (templates, prompts, schemas, config, your `custom/` overrides) and `.claude/` (the `/alterspec-*` skills and the four agents).
- 2 Open Claude Code in the project and type `/alterspec-init`. Claude interviews you, at most three questions at a time: the product and its problems, the apps and channels, personas and roles, the business areas (modules, each with a 2–6 letter code; shared screens go in GLB), and the key terms with the words not to use.
- 3 It then writes `spec/application/`: application, personas and roles, glossary, and one `module.md` per module. Every object is created by the CLI (`alterspec new ...`), which picks the ID, the file and the template.
- 4 Finish with `/alterspec-validate`. The spec is readable Markdown with YAML front-matter; you can edit it by hand at any time, the linter tells you when something no longer fits.

What exists after this step: the skeleton. No entities, capabilities or screens yet.

2. Tuning the spec during initiation

The first version is built before any baseline exists, so edits go straight into `spec/`.

- `/alterspec-create-entity NAME Title: a business thing with attributes (business kinds only: text, amount, date, choice, reference...), relationships and a lifecycle (states and allowed transitions).`
- `/alterspec-create-capability MOD "Title": one user goal, one session, the roles allowed to perform it. The interview walks through the thirteen sections (user story, value, preconditions, main flow per screen and action, exceptions, data in and out, rules, state transitions, notifications, permissions, acceptance criteria, out of scope, open questions). The analyst agent drafts the body from your answers only; it never invents facts.`
- `/alterspec-create-screen MOD "Title": what people see and do, in business terms. fields lists the data shown per entity (attribute names exactly as the entity has them, in list, view or edit mode); actions name the capability each performs.`
- `/alterspec-refine ID: closes the gaps in one object. It merges the linter's findings with the analyst's gap analysis, asks only about those, and moves the object to refined when nothing is missing. ready and beyond always need your explicit word.`
- `/alterspec-validate: the 41 deterministic rules plus the reviewer agent's semantic findings (contradictions, data nobody produces, permission holes, untestable criteria) in one report. Fix errors as they appear; it is far cheaper than later.`

Tuning never means guessing: when you don't know yet, the answer is recorded as an open question (`DEC-*`) against the object, and the object cannot become `refined` until it is answered.

When the first version is agreed: `npx alterspec baseline`. From then on every edit is a change proposal (section 5).

3. Wireframe and experience: making and tuning the UI

Two layers show the product. Think of them as a dry and a wet signal.

Dry: the generic wireframe. `alterspec views` writes `spec/_generated/wireframe/`: one plain page per screen, built only from the business spec (fields, actions, roles, entry points, the empty / no-permission / validation states) with made-up data. Anything the spec doesn't say shows as a highlighted "Not specified" note. You never edit it; it is always current, and it is what the experience layer is measured against.

Wet: the experience layer (`spec/experience/`), the future app:

- `design-system.md` (tokens, components), `patterns.md` (page archetypes and their regions).
- `screens/UX-SCR-...md`: the UX contract of one screen. Every business element with its region, component and exact label; every state as a link (`?as=<role>&state=<id>`); interactions, validation messages, loading, errors, responsive and accessibility behaviour.
- `mockups/`: a working app. `index.html` signs you in as a demo person; `SCR-...html` pages are bound to demo data kept in the browser (lists open records, forms save, actions change data the way their capability says); `config.js` (name, logo, currency, demo people) and `data.js` (demo data) are yours; `kit/` is the runtime.

How you change the UI:

- 1 `/alterspec-experience init` once: the app, its name and brand, demo people, realistic data.
- 2 `/alterspec-experience SCR-XX-NN` per screen: the CLI drafts the contract and the page so that they already pass every check; you answer what the draft cannot know (archetype, placement, components, labels, what happens after each action); the UX designer agent shapes both files. Open `spec/experience/mockups/index.html`, sign in as each role, try it, iterate.
- 3 `/alterspec-experience review SCR-XX-NN`: the experience reviewer agent produces a parity table (business spec, contract, mockup, verdict per element). Only a clean table lets `experience reviewed` record the review.
- 4 Edit the page or the contract by hand whenever you like. Keep `data-src`, `data-roles` and `data-show-in`; the checks read them. Design-only elements (icons, separators, help text that restates the spec) are free. Your edits are never overwritten: `rebuild` replaces a page only while it is the untouched draft; afterwards it keeps the page, writes the fresh render to `_generated/experience/rebuild/` and reports what differs: business content the spec lacks (goes through the change flow), check findings, or an outdated kit to merge by hand.

How everything stays in sync:

- Every business element of a screen has a stable marker, `data-src="SCR-PRC-01.A01"`. The contract must list each marker exactly once and nothing else; the page must carry each marker exactly once and nothing else; the text inside must contain the declared label; narrowed elements must carry the right `data-roles`; every declared state must be marked `data-show-in`. These are lint errors, not warnings.
- Each contract records the fingerprint of the dry page it was aligned with. When the business screen changes, the linter warns `experience-stale`; `alterspec experience sync SCR-...` removes dropped elements, adds new ones as drafts to place, adds states and default views for new roles, updates who sees what, and records the new alignment. Each contract also records the fingerprint of itself and its page at the last clean review; any later edit warns `experience-unreviewed`.
- If the design needs something the business spec lacks (a field, a filter, a message), it goes into the business spec first, then `sync`. Never only into the mockup: the checks would reject it.

4. Why spec, wireframe and experience cannot drift apart

- One source of truth: capability, screen and entity front-matter. Module lists, role matrices, screen back-references, entity coverage, traceability, the wireframe and the data the mockups load (`spec.js`) are all generated from it. Generated blocks carry a hash; a hand edit is an error.

- One vocabulary: IDs are stable and never reused; every reference is checked to exist; glossary synonyms and technology words are flagged.
- Closed loops: every entity transition is performed by some capability; every consumed event has an emitter; every flow step is a capability and every capability is in a flow; every screen action is a capability one of the screen's roles can perform; edited data on a screen is created or updated by one of its actions.
- The experience layer realises exactly the dry page (section 3), and a business screen can't go to review in a change until its experience follows.
- Handoff refuses anything with lint errors, any capability below `ready` (unless you say `--allow-draft`), and any screen whose experience is not ready, reviewed and finding-free. There is no override for the last one: developers only ever receive aligned screens.

5. Changing an existing feature

After the baseline, the main `spec/` is read-only for people. Direct edits are `direct-edit` errors.

- 1 Start or pick a change: `/alterspec-change "Why we change this"`, or let `/alterspec-change-capability CAP-... (-screen, -entity, -module)` handle one object and ask you which change to use.
- 2 Every touched object is copied into `spec/changes/CHG-NNN/spec/` at its normal path (`alterspec change edit`), new objects are created there (`alterspec new ... --change CHG-NNN`), removals are recorded (`alterspec change remove`). The copy remembers the fingerprint it started from.
- 3 Work inside the change: edit the copies, run `alterspec validate --change CHG-NNN` (the spec as it would be after the change), `alterspec experience sync SCR-... --change CHG-NNN` for affected screens, and `/alterspec-impact CHG-NNN` for what is affected: flows, acceptance criteria, screens, roles, generated views, new and resolved findings, plus the reviewer agent's semantic findings.
- 4 `alterspec change status CHG-NNN in_review` is refused while the change has errors, conflicts or no content.
- 5 `/alterspec-apply CHG-NNN` shows the impact and asks for your approval by ID ("approve CHG-NNN"). Nothing else counts. Then it merges, raises `version` on every modified object, regenerates views, updates the baseline and archives the proposal.

Two changes in flight never get the same new ID. If one is applied and touches an object the other also edits, the second gets a conflict and must re-read the object and `change edit --rebase`. A change edited after approval has to be approved again.

6. Creating a new feature

A new feature is a change proposal that mostly adds:

- 1 `/alterspec-change "New feature"`; describe why and what, in business language.
- 2 Inside it, the same commands as during initiation, all with `--change CHG-NNN` behind the scenes: new entity or attributes, new capabilities (one goal, one session each), new or changed screens, rules, events, a flow step that ties the feature into an existing journey.
- 3 `/alterspec-refine` the new objects until they are refined; `/alterspec-experience` their screens.
- 4 `/alterspec-impact`, `review`, `in_review`, `/alterspec-apply` with your explicit approval.
- 5 When it is ready and its screens are reviewed: `/alterspec-handoff CAP-... <target>` exports to Spec Kit, OpenSpec, BMAD or a self-contained bundle (with the wireframe and the experience of its screens) into `handoff/<target>/<ID>/`, with a manifest of the exact versions handed over.

Before the baseline, the same feature is simply created directly, without a change.

7. Deterministic parts vs. model parts

Deterministic (the CLI, same input -> same output)	Model (Claude Code skills and agents)
IDs, file locations, templates (new)	Interviewing you, three questions at a time
Schema and the 41 lint rules (validate)	Semantic review: contradictions, gaps, permission holes (reviewer agent)
Generated blocks, matrices, traceability, wireframe, spec.js (views)	Drafting body text from your answers (analyst agent)
Fingerprints, baseline, conflicts, approval hash, merge, versions (change, impact, apply)	Writing the "Why" and "What changes" of a proposal
Experience drafts, sync, dry and review fingerprints, parity checks	Shaping layout, components, labels, realistic data (UX designer agent)
Handoff rendering and the experience gate	Parity table and developer questions (experience reviewer agent)
Doctor, install, update	Optional polish of handoff wording (never content)

Rules that keep the two apart: the model never picks an ID, copies a template, edits a GENERATED block or runs `baseline` or `apply` on its own. Skills that change the spec run only when you type them; the model may run `validate`, `views` and `impact` by itself because they are read-only or regenerate derived output. Approval of a change is always your explicit word naming the change. Reviewer agents are read-only by construction (no Write or Edit tools). Everything deterministic can run in CI (`validate`, `views --check`) without Claude.

8. Updating alterspec

```
npm install --save-dev @alterset/alterspec@latest
npx alterspec update           # refreshes .alterspec/ and the .claude/ commands; never touches spec/, co
npx alterspec views           # regenerates derived output; 0.6 moved the wireframe from _generated/prot
npx alterspec doctor
```

Commands you edited under `.claude/` are skipped by `update` unless you pass `--force`; customisations belong in `.alterspec/custom/` instead, where updates never reach.

Quick reference

```
Start      npx alterspec init --name "X"          /alterspec-init
Author     /alterspec-create-entity|capability|screen|module      /alterspec-refine ID
Check      /alterspec-validate      npx alterspec validate [--json] [--change CHG]      npx alterspec view
Design     /alterspec-experience init | SCR | review SCR | sync SCR | rebuild SCR
Freeze     npx alterspec baseline
Change     /alterspec-change "title" | /alterspec-change-capability CAP      /alterspec-impact CHG      /alter
Deliver    /alterspec-handoff CAP|MOD bundle|speckit|openspec|bmad|all
Health     npx alterspec doctor      npx alterspec show ID
Update     npm i -D @alterset/alterspec@latest && npx alterspec update && npx alterspec views
```